

TASK 1:

Sub Task 1:

Sparse Optical Flow:

Problem: To implement sparse optical flow using Lucas-Kanade Algorithm

Optical flow refers to the flow lines of an object moving against its backgrounds which tracks how various points on the object move with respect to the background in order to track it.

To implement optical flow, we take the basic idea from Lucas Kanade, which is that the intensity of a point across 2 different frames does not change or, changes very little (not enough to make a difference in this case)

Therefore, taking this assumption, we can say that:

$$\Delta I = 0$$

$$\implies I_x * u + I_y * v + I_t = 0$$

where (u, v) is the vector of motion of the point.

Now, obviously here we have one equation and 2 variables, so this is not solvable. However, what we do is we take a grid of pixels (typically 5×5) and group them together, saying they have the same (u, v) motion vector. Then, (u, v) is calculated using the principle of least squares from these multiple equations, as we have an over constrained system of equations.

Knowing (u, v) for a set of pixels, we can trace streamlines for that object which will help us to track it.

Taking this to the next level, we can further fine-tune Lucas Kanade so that it tracks motion to a high degree of accuracy, which is by using the concept of pyramids.

Pyramids is an idea where basically the image is first downscaled heavily, such that only the most prominent features are tracked using Lucas Kanade. It then

upscales the image one resolution step at a time, and applies Lucas Kanade on the difference between the actual motion and the scaled up version of the motion captured in the previous iteration, so that it tracks the finer details instead of again tracking the same dominant motion features. This technique is also known as warping.

To know which points to track, `cv2.goodFeaturesToTrack()` method was used, which uses the Shi-Tomasi algorithm to detect corners in an image, and considers these points as the best to track. This completely avoids the aperture problem as well, which is when motion is seen from an edge, in which case motion parallel to the edge remains undetected and direction of motion cannot be figured out.

To optimize this, a lot of the calculations were vectorized and Sobel matrices were used, which dramatically sped up the computation time. Also to further clean the output, a magnitude filter was added, which tracked the magnitude of motion of objects and whether it was relevant enough to be added to the final result or not.

Before computing gradients, gaussian blurring was also done to smoothen out the binary image so that taking derivatives would not be an issue.

Temporal smoothing, i.e, using flow from the previous frame to join over into the next frame was also added to increase the visual quality.

Initially, I had a lot of noisy flow vectors which took up a lot of the screen and did not help much in the tracking. In order to fix this, I switched to streamlines which joined up all the flow vectors to produce fluid-flow like visual output, while simultaneously aggressively cutting down on noise to further enhance quality.

Dense Optical Flow:

The implementation for dense optical flow was very similar to that of sparse optical flow, which very similar fine tuning and adjustments, except for the fact that in dense optical flow, we track the motion of all points instead of just corners/`cv2.goodFeaturesToTrack()`, which is a big upgrade in obstacle detection.

All that was changed was not using `cv2.goodFeaturesToTrack()` and instead just running Lucas-Kanade on every single point.

Sub Task 2:

1. Introduction

This solution implements a real-time autonomous navigation system for obstacle avoidance using a monocular camera. The system relies on sparse optical flow to estimate motion in the scene, computes Time-to-Collision (TTC) for detected feature points, and generates steering commands using a potential field planner combined with a commitment-based decision layer and a sliding mode controller. The vehicle motion is simulated using a bicycle kinematic model.

2. System Pipeline Overview

The system operates as a sequential pipeline:

Image Acquisition → Region of Interest → Feature Detection → Optical Flow
→ Time-to-Collision Estimation → Danger Assessment → Obstacle Memory
→ Potential Field Planning → Commitment Decision Logic
→ Sliding Mode Control → Vehicle Kinematics → Speed Control → Recovery Logic

Each module is described in detail below.

3. Image Acquisition and Region of Interest

The system captures frames from a forward-facing monocular camera. Instead of processing the entire image, a trapezoidal Region of Interest (ROI) corresponding to the road area is selected. This allows for removal of unnecessary background noise, which causes interference with obstacle detection and FOE calculations. The Region of Interest is made trapezoidal as that is the shape of the road if the car is in the centre of the road and looking straight ahead. The Region of Interest also blocks out unnecessary noise from the road boundaries.

4. Feature Detection and Sparse Optical Flow

I have used sparse optical flow, implemented in subtask 1 and Otsu thresholding for obstacle detection, where `cv2.goodFeaturesToTrack()` gives the points to track and observe. Gradient smoothing was applied in order to smoothen out the binary image so that further computation can be done on it. In order to detect obstacles and not the road boundaries(which had a stronger presence), a smoothening out of the road was done to blur any sharp features and to give the obstacle higher priority. I also increased the minimum size of a detection to be counted as an obstacle to 150 pixels, which further eliminates road boundary noise. Also, points

in the bottom part of the screen were all clipped from the obstacle detection part so that again the highlight would be the actual obstacle and not the road.

5. Time-to-Collision (TTC) Estimation

Time-to-Collision is estimated from optical flow divergence. Points that expand outward quickly in the image correspond to approaching objects.

Conceptually:

$$TTC_i = \frac{\sqrt{(x - x_{FOE})^2 + (y - y_{FOE})^2}}{\sqrt{v_x^2 + v_y^2}} \quad (1)$$

where v_x , v_y are the velocities of the pixel and x_{FOE} , y_{FOE} are the coordinates of the Focus of Expansion. In monocular vision, TTC is approximated using image motion rather than true depth.

Interpretation:

TTC	Meaning
Large TTC	Object far away
Small TTC	Object close / imminent collision

TTC is the primary predictive variable used in obstacle avoidance.

6. Danger Detection

For danger detection, the point with the lowest TTC(Time to Collide) is taken as reference and a window of points around it are taken to be cluster which will force the commitment direction into one of the lanes. According to the TTC, danger is prioritised such as the road boundaries being higher priority at a higher TTC and the obstacle being a higher priority at a lower TTC. A cool down period was also there in vehicle steering, where after crossing an obstacle, it would just go straight and only to object detection and not react responsively to it but to store that information. Again, this was done to stabilize the object and clamp its oscillatory behaviour.

7. Obstacle Memory (Temporal Fusion)

Due to the vehicle's motion, a lot of the time it oscillates a bit back and forth due to random noise from optical flow. In these kinds of situations, it may lose track of the current obstacle and start prioritising road boundaries or later obstacles, causing it to violently swerve back into a completely different trajectory, making the vehicle uncontrollable. In order to fix this, an obstacle memory was added, which remembers which lane to commit to, to disable road forces when the object is between the road and the obstacle so that the car can pass smoothly instead of being shoved around. Obstacle memory was also flushed periodically after it crossed an obstacle to prevent any memory errors when it went for the next obstacle. Frames were also deleted from memory if there was no imminent danger in them or if the next frame did not have that danger any longer.

8. Potential Field Planning

The vehicle is guided using a potential field approach consisting of:

$$F = F_{\text{repulsive}} + F_{\text{attractive}} + F_{\text{road}}$$

8.1 Repulsive Force (Obstacle Avoidance)

Repulsive force pushes the vehicle away from obstacles. The way to calculate repulsive force is to take the summation of all the gradient vectors and then divide it by the TTC. This makes sense intuitively as the bigger the obstacle, the more points it has and the closer the obstacle, the higher the force should be.

8.2 Attractive Force (Goal Seeking)

An attractive force pulls the vehicle forward towards the goal always. There is another additional force that I added that pulls the vehicle into the centre of a lane once it has committed to a lane so that it doesn't oscillate wildly.

8.3 Road Boundary Force

A boundary potential prevents the vehicle from leaving the road by applying a strong repulsive force near the road edges. The boundary potential works like a damped spring, where the closer you get to the wall, the stronger the force will be.

The push-back is damped so that it doesn't force the vehicle to oversteer and reverse directions, crashing into the obstacle.

A potential field is just the most basic implementation of forces but it is not enough as minor perturbations in the system cause the car to wildly oscillate. Therefore, forces such as lane centre attracting force, damped spring wall force are important to further stabilize the trajectory of the vehicle and reduce oscillatory behaviour, as that really messes with obstacle detection.

9. Commitment Decision Logic

Reactive potential fields can cause oscillations when multiple obstacles are present. To solve this, the system includes a **commitment mechanism**.

The commitment mechanism works as follows : any "obstacle" that has been detected in one frame is given some weightage of being an obstacle versus being some random garbage noise. This data is then averaged over a couple of frames, and if clears a threshold of being an obstacle, it is counted as such and the lane opposite to that obstacle is the one which the vehicle tries to lock itself into. There are other conditions for commitment as well, such as a TTC barrier, along with the fact that once it commits, it can't keep changing its decision(ideally) and so would stick to its path.

10. Spatial Gating

When the vehicle finally commits to a direction, a gating is applied which prevents it from seeing obstacles in the lane its committed to, as otherwise it would detect those as obstacles and again reverse its direction, causing the unwanted oscillatory behaviour.

11. Sliding Mode Control (SMC)

This was basically implemented exactly as it was given in the paper, no changes.

12. Speed Control

The vehicle being static was a huge issue, as it would fail to turn in time, or it would oversteer itself off the road. Therefore, during turning, I introduced a feature where when the vehicle is committing/turning/avoiding an obstacle, it dynamically

reduces its speed in order to allow itself a chance to make the turn properly and not undershoot or overshoot itself.

13. Stall Detection and Recovery

In some cases, the vehicle would not start up at all or would be moving at a snail's pace. In such cases, the steering and controls had to be overridden and the vehicle jumpstarted back to life for a couple of frames, after which regular tracking and control would take over.

Problems Encountered:

At one point, I also encountered the issue of ghost obstacles, where it was stroing every single point detected by `goodFeaturesToTrack()` as an obstacle and stroiung that for future use even though the vehicle as already passed it, which completely filled up that memory and caused the vehicle to chase after those "ghost" obstacles.

I also removed all 3 white lines on the road as they were messing with my obstacle detection and i had a much stronger damped spring force system for the road boundaries anyways.

I also tried dividing up the camera feed into sectors and figure out which one had the most obstacle-like features, which I would then steer against. this too failed because of the road boundaries being very good features to track and completely overshadowing the obstacle.

When I used just potential, the car also tried to squeeze itself between the obstacle and the road boundary on the wrong lane. To fix this, I increased the wall force tremendously while simultaneously making a gap filter for the vehicle, below which it wouldnt pass itself through.

Since the road was quite smooth and textureless, Shi-Tomasi had difficulties getting any points at all, and I was left wondering why my trajectories seemed so random, but in reality it was just the walls giving forces from either side with the obstacles playing 0 role. Upon lowering the criterion for a point to be good, a lot more points were tracked and the trajectory was finally being affected properly by the obstacles instead of just being pushed around by the wall. I also used `cv2.inbulit` optical flow function in order to help me debug, which pointed out my tracking as being the problem.

A lot of the time, danger was 0 and TTC was TTC_MAX, even though the obstacle was right in front, because the vehicle was oscillating so much that the obstacle wasn't in proper view for more than 2 frames and my conditions were too harsh like it being counted as an obstacle only if it was found in 6 consecutive frames so that parameter had to be lowered.

Conclusion:

This is the most polished version so far, there have been versions where the car is attracted instead of repelled by the obstacle, the car has done a backflip for some reason, wandered off into the distance after oversteering itself off the road. Most problems were just cyclic, where solving one led to another in a vicious cycle. This was my best attempt at task 1, signing off from here.